

Seven Design Decisions that Apache Cassandra's Successor is Built On

ScyllaDB is the next-generation Apache Cassandra compatible database. Its user list of IBM, Zenly, Snapfish, AdGear, etc, is in itself a testament to its excellence. With an installer for every platform, the reader has an open invitation to give it a try.



Loved by its community of users, Apache Cassandra is widely accepted as the best highly available distributed database. With roots in both Dynamo and Bigtable, it represents 15 years of progress. However, while Cassandra is known for flexible replication, multi-data centre support and gigantic homogeneous clusters, its many issues are well known too. At times, one feels that Cassandra is a victim of its own success.

At ScyllaDB, we set our sights on delivering an open source alternative to Cassandra — one that's rebuilt from the ground up, to deliver higher throughput, to maintain consistently low latencies and to reduce the time users spend tuning their databases. At the same time, we wanted to preserve all the things the Cassandra community loves.

Once we'd decided what we wanted to build, we had to make several important decisions about how we'd go about building it. Here, I'll share the seven fundamental design decisions we made when architecting a better performing Cassandra and the results those decisions have had on our open source project.

Design decision 1: C++ instead of Java

We didn't have to think long before making this initial design decision. Most systems professionals understand that Java isn't a good language for systems programming. Why? Because it deprives the user of control. A modern database requires the ability to use large amounts of memory and to have precise control over what the computer is doing at any time. Java isn't

well suited to address either of these requirements. However, C++ serves both purposes well. It not only gives developers very precise control over everything that they might want to do, but it also allows the creation of abstractions, which lets you create complex code in a manageable way.

Our initial analysis of Cassandra revealed the problems caused by its use of Java. The lack of control makes it hard for Cassandra developers to make the database do what they want it to. For instance, consider the recent kernel-level API enhancement, sponsored by Scylla, to improve the kernel polling interface. You can't do that with a Java virtual machine (JVM). Scylla developers check the assembly generated code frequently and verify efficiency metrics to look for potential optimisations.

JVM's performance and latency issues caused by garbage collection are well documented. Cassandra's developers try to bypass the garbage collector by using off-heap data structures. However, the memory becomes fragmented and is harder to tune, defeating the whole purpose of memory being managed by the runtime. By deciding to build in C++, we avoided these problems altogether.

Design decision 2: Compatibility

We realised at the outset that the Cassandra user community isn't interested in learning yet another database or another data model. What they wanted instead was a drop-in Cassandra alternative, which overcomes the performance and latency issues that had hindered their applications. So rather than build